

CASE STUDY · MAY 2026

PartSelect Chat Agent

A production-grade customer-support agent for refrigerator and dishwasher parts.

PREPARED FOR **INSTALILY** · SARTHAK CHANDARANA

Appliance parts support is slow, expensive, and brittle.

Live agent contacts cost \$5 to \$8 each. Most are repeat questions:

will this part fit my model?

Compatibility uncertainty is the top driver of cart abandonment on parts

e-commerce. Customers leave when they can't get a confident *yes* or *no*.

Returns from "wrong part" purchases compound the cost with shipping and restocking, and erode brand trust.

FOUR QUESTION PATTERNS THAT MUST BE ANSWERED CORRECTLY AND EXPLAIN THEMSELVES

Install	<i>How do I install PS11752778?</i>
Compatibility	<i>Does this part fit my model?</i>
Troubleshoot	<i>My ice maker is broken, what part?</i>
Compound	<i>Symptom + model + install in one turn.</i>

If the compound query works, the others fall out for free.

"Ice maker on my Whirlpool WRF555SDFZ is broken — what part do I need, and how do I install it?"

One user message. Four tool calls in a single turn:

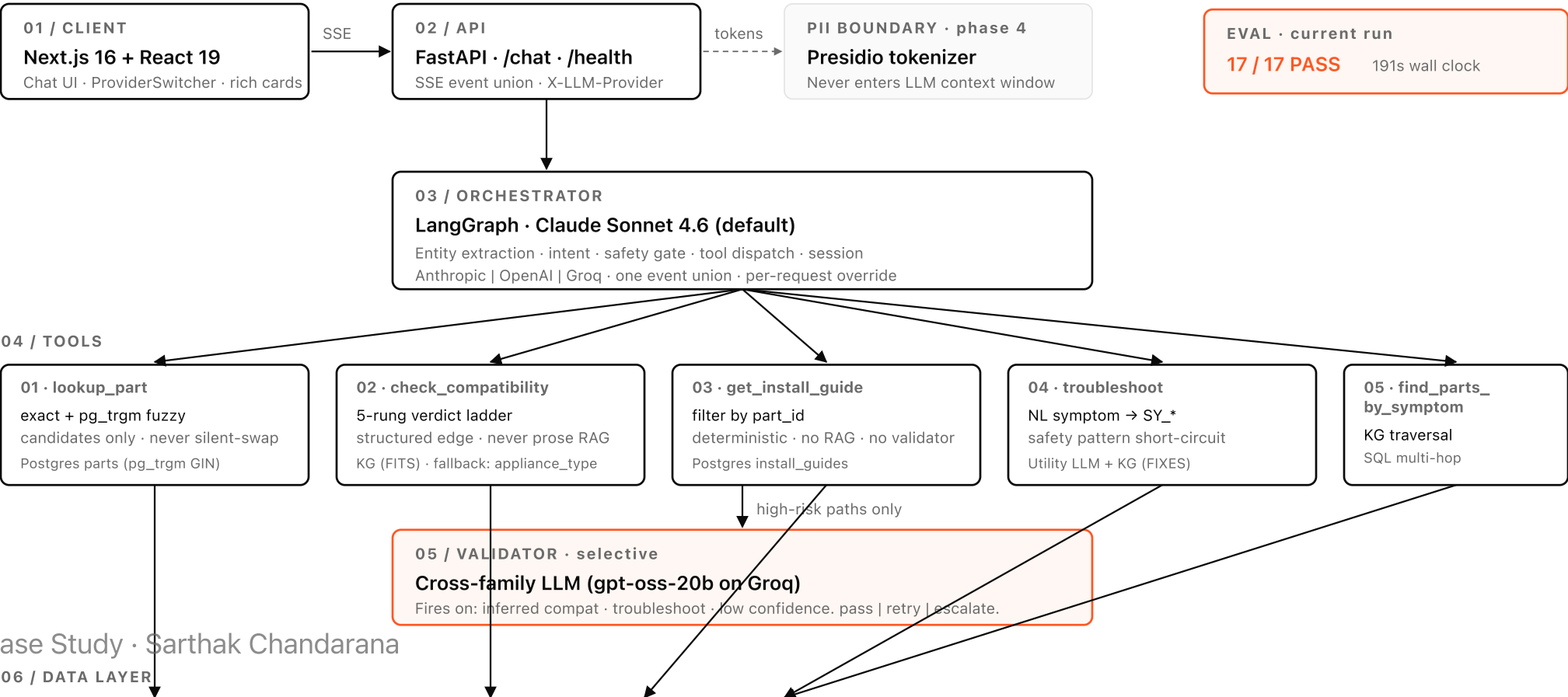
01	troubleshoot	Natural language → canonical symptom → ranked candidates
02	find_parts_by_symptom	KG traversal, annotated with model fitment
03	check_compatibility	Structured edge verdict
04	get_install_guide	Deterministic step-by-step

Verdict **YES**. Validator gates the recommendation. UI renders product + compat + install in one bubble.

One orchestrator. Five typed tools. Selective validator.

PartSelect Chat Agent — System Architecture

Phase 1 POC. All five tools built. 17 / 17 eval. Hard rules enforced end to end.



Orchestrator + typed tools, not router-to-specialists.

Why this shape:

- Every action is a typed Python function with a JSON Schema contract.
- The reviewer reads `tools/registry.py` and knows the entire action surface.
- No tool calls an LLM internally. Auditable, fast, regression-proof.
- Single conversational thread means session memory (`last_part` , `brand` , `model`) carries across turns without specialist coordination.

Tradeoff named:

Less *agentic*. We do not get emergent multi-agent behavior. For this domain that is the right trade: the failure cost of a hallucinated compatibility verdict outweighs the upside of a more creative agent.

"compatibility verdicts come from edges; prose inference is a marked, lower-confidence fallback."

Knowledge graph as a structured backbone. Not GraphRAG.

Compatibility is a structured edge lookup. It is never LLM reasoning over prose.

The verdict ladder (in order):

1	Missing entity	unknown · low
2	KG edge exists	yes · high
3	Cross-appliance	no · high
4	Series-hint match	inferred · medium
5	No edge	no · medium

The trick question: is PS11752778 (refrigerator part) compatible with WDT780SAEM1 (dishwasher)?

Most agents say "no, I don't see a fitment". That's technically correct but unhelpful.

Our agent says:

*"PS11752778 is not compatible with your WDT780SAEM1, and the reason is an **appliance type mismatch**. PS11752778 is a refrigerator ice maker assembly. WDT780SAEM1 is a dishwasher."*

That's rung 3 firing. Inference path (rung 4) is marked and gated by the validator.

Six hard rules. Enforced in code, verified in the eval.

01	No em dashes in any user-facing text.	eval asserts U+2014 absent in every reply
02	PII never enters the LLM context.	Presidio inbound · ticket form bypass · log redaction
03	Compatibility verdicts from structured edges.	verdict ladder; prose inference marked + gated
04	Safety symptoms short-circuit to escalation.	regex pre-empts LLM and KG
05	Fuzzy SKU matches require user confirmation.	repo returns <code>list[Part]</code> ; never <code>Part</code>
06	Validator never silently passes a failure.	<code>pass retry escalate</code> , retry cap of 1

These are the failure modes that get a vendor fired. Getting them right is what unblocks enterprise sales.

Provider-agnostic LLM gateway. Insurance, not lock-in.

One normalized event union:

```
TextDelta | ToolCall* | Usage | Done | StreamError
```

Same orchestrator code runs on Anthropic, OpenAI, and Groq behind a single `LLMProvider` protocol.

Per-request `X-LLM-Provider` header flips providers mid-conversation. The frontend `ProviderSwitcher` writes it; the reviewer can flip live in the demo.

Insurance benefits:

- **Rate-limit resilience.** Groq 429 in production? Switch to Anthropic; user never sees it.
- **Cross-family validator.** Validator uses a different family from the orchestrator. Diversity catches errors one family makes that another does not.
- **Cost flexibility.** Cheap Groq for utility role (symptom mapping); premium for orchestrator.
- **No vendor lock-in.** A new model from a new provider is one file.

17 / 17 on the case-study eval.

PASS RATE BY CATEGORY

CATEGORY	CASES	RESULT
Install	2	PASS
Compatibility (all 5 verdict rungs)	5	PASS
Troubleshoot	1	PASS
Compound (the real test)	1	PASS
Edge (fuzzy, case, missing, safety)	4	PASS
Out-of-scope	2	PASS
Adversarial (prompt injection)	1	PASS
Validator (selective trigger)	1	PASS

METHODOLOGY

Deterministic checks, not LLM-as-judge. Every assertion is a Python predicate over tool payloads and final text. Regressions are actionable: a failed `p.verdict == 'yes'` tells you exactly which row in the ladder broke.

191 seconds wall clock total (Anthropic orchestrator + Groq validator).

Every reply printed verbatim in the eval report so fluency and tone are reviewable too.

[docs/eval_results.md](#)

A focused agent unlocks three categories of value.

DEFLECTION

~\$1.8M

illustrative annual operating cost reduction at 50k/month contacts × 60% deflection × \$6 saved per deflected contact.

CONVERSION LIFT

A confident, **explained** yes/no/inferred compatibility verdict directly recovers cart abandonment from compatibility uncertainty — the largest single contributor on parts sites.

ENTERPRISE-SALES UNLOCK

Pll never leaves the boundary. Safety symptoms always escalate. These are the failure modes that get a vendor *removed* from a procurement shortlist.

Getting them right is **table stakes for closing PartSelect-scale customers**, and they are already enforced in the POC.

TIME TO VALUE

50 parts seeded synthetically. A real catalog ingest swaps the data, not the architecture. Phase 2 is *scrape-and-load*, not *rebuild*.

Architecture has explicit seams. Each phase is a drop-in.

PHASE	WHAT	HOW IT SLOTS IN
02	Real PartSelect scrape + hybrid RAG (BM25 + dense + RRF + rerank) over repair stories.	VectorStore protocol → Chroma → pgvector swap. Scraper hits same Postgres schema.
03	Validator expansion: every troubleshoot recommendation and every inferred compat now grades.	One conditional edge in <code>graph.py</code> . Validator already built.
04	Production hardening: per-IP rate limit, provider fallback chain, Neo4j swap, LangSmith tracing, PagerDuty on hard-rule violations.	KnowledgeGraph protocol → Neo4j is one file. FastAPI is stateless; session in Postgres; KG snapshot loads in <10ms.
05	Hosted demo on Vercel (frontend) + Fly.io (backend) + Supabase pooler.	Already env-driven. <code>NEXT_PUBLIC_BACKEND_URL</code> + <code>CORS_ORIGINS</code> .

THANKS FOR READING

Happy to walk through the code live.

Source · README · run instructions · eval report · architecture doc

github.com/sarthakc123/instalily-partselect-agent

SARTHAK CHANDARANA · SARTHAK.CHANDARANA99@GMAIL.COM